PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

Title of Project: Parallel Image Processing



Name	Registration Number
Hoor-ul-ain Wajid	B22F0068DS004
Maheen Shahid	B22F0944DS025
Kinza Mazhar	B22F0591DS007

Batch: Fall 2022

Submitted To: Ms. Faiza Bukhari

Introduction:

Image processing involves operations like filtering, enhancement, and edge detection. These operations are computationally expensive for large, high-resolution images. To speed up these tasks, we implemented a **hybrid MPI + OpenMP parallel processing system**, where:

- **MPI (Message Passing Interface)** distributes image rows across multiple processes running on different cores or machines.
- **OpenMP** is used inside each process to parallelize pixel-level operations using multi-threading.

The system supports both **PNG and PPM formats**. PNG images are automatically decoded to PPM for processing, and the result can be saved back as PNG.

Objectives:

The main objectives of the project are:

1. **Hybrid Parallel-Distributed Architecture:** Implement two-level parallelism combining distributed computing (MPI) and parallel computing (OpenMP)
2. **Format Conversion**: Support both PNG and PPM image formats with automatic conversion
3. **Multiple Filter Operations**:
   - Grayscale conversion
   - Gaussian blur (5x5 kernel)
   - Edge detection (Sobel operator)
   - Binary thresholding
   - Image sharpening
   - Brightness enhancement
   - Contrast enhancement
   - Color quantization compression
4. **Performance Monitoring**: Track processing time and display progress bars
5. **Scalability**: Handle images of any size with dynamic workload distribution

Tools and Technologies:

1. **Language:** C++
2. **Parallel and Distributed Libraries:**
   - MPI (MS-MPI)
   - OpenMP
3. **Image Library:** LodePNG (for PNG encoding/decoding)
4. **IDE / Build Tools:** Visual Studio (MSVC compiler)
5. **Execution:** mpiexec command for running MPI programs

Implementation:

Execution Flow

Step 1: MPI Distribution (Coarse-Grained)

- Rank 0 reads full image
- MPI_Bcast sends dimensions to all processes
- MPI_Scatterv distributes row chunks to processes
- Each process gets contiguous rows (e.g., 0-359, 360-719, etc.)

Step 2: OpenMP Processing (Fine-Grained)

- Within each MPI process, OpenMP threads spawn
- Threads divide their process's rows further
- All threads access shared local Img array
- Filters apply in parallel (e.g., blur, sharpen)

Step 3: MPI Collection

- MPI_Gatherv collects processed chunks back to rank 0
- Rank 0 writes final image to disk

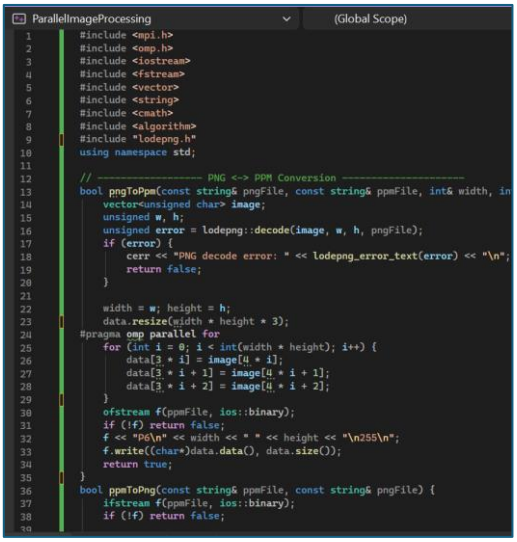
MPI Functions

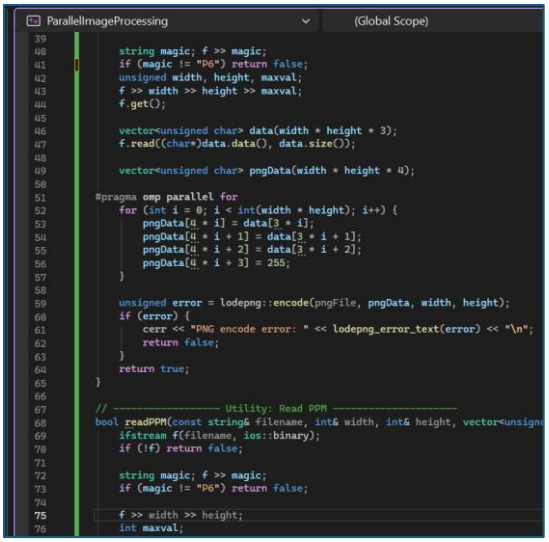
Function	Location	Purpose	Distributed Concept
MPI_Init()	main()	Initializes MPI environment	**Process Spawning** - Start parallel execution
MPI_Comm_rank()	main()	Gets process ID (0, 1, 2...)	**Process Identification** - Know who you are
MPI_Comm_size()	main()	Gets total process count	**Process Discovery** - Know team size
MPI_Bcast()	main()	Broadcasts data from one to all	**One-to-All Communication** - Collective operation
MPI_Scatterv()	main()	Distributes data chunks to processes	**Data Distribution** - Domain decomposition
MPI_Gatherv()	main()	Collects results from all processes	**Data Collection** - Result aggregation
MPI_Wtime()	main()	High-resolution timer	**Performance Measurement** - Benchmarking
MPI_Finalize()	main()	Cleans up MPI environment	**Process Termination** - Graceful shutdown
MPI_Abort()	Error handling	Terminates all processes	**Fault Handling** - Emergency stop

OpenMP Clause/Functions:

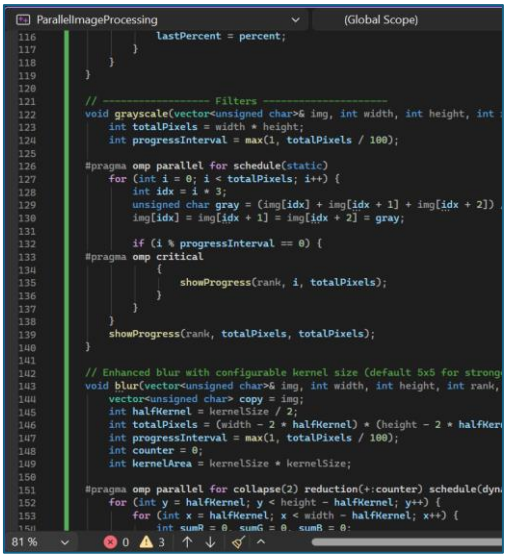
Clause/Function	Location	Purpose	Parallel Concept
#pragma omp parallel for	PNG/PPM conversion	Splits loop iterations among threads	**Data Parallelism** - Dividing data across threads
omp_get_max_threads()	main()	Gets available thread count	**Resource Discovery** - Query system capabilities
schedule(static)	grayscale(), brightness(), contrast(), compress(), binaryThreshold()	Pre-assigns equal chunks to threads	**Static Load Balancing** - Fixed work distribution
schedule(dynamic)	blur()	Assigns work to threads at runtime	**Dynamic Load Balancing** - Adaptive work distribution
collapse(2)	blur(), edgeDetect(), sharpen()	Merges nested loops for more parallelism	**Loop Fusion** - Increases parallel work units
reduction(+:counter)	blur(), edgeDetect(), sharpen()	Safely combines thread-local counters	**Reduction Operation** - Safe aggregation across threads
#pragma omp critical	All filters (progress updates)	Ensures only one thread executes block	**Mutual Exclusion** - Prevents race conditions

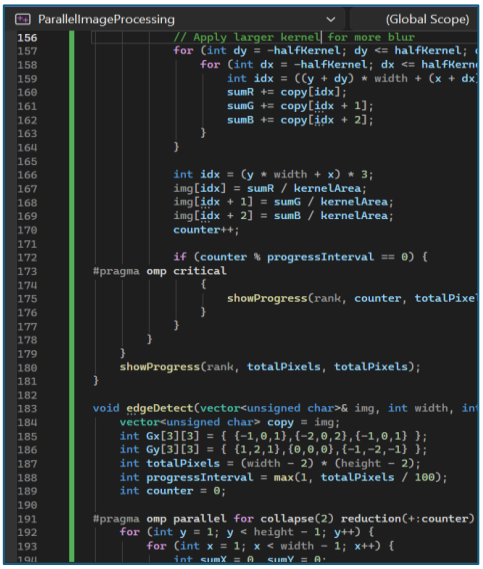
Code:

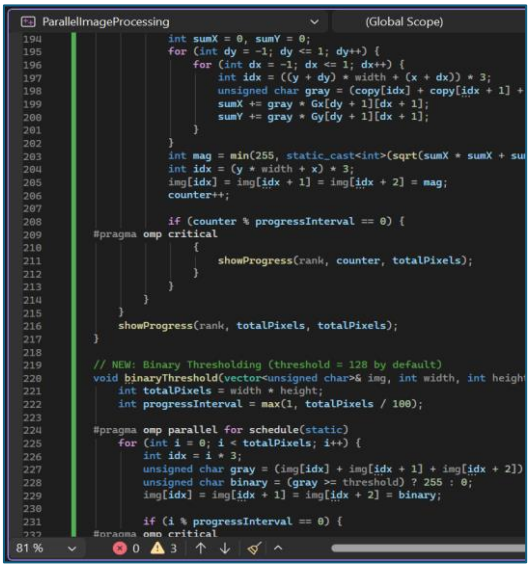


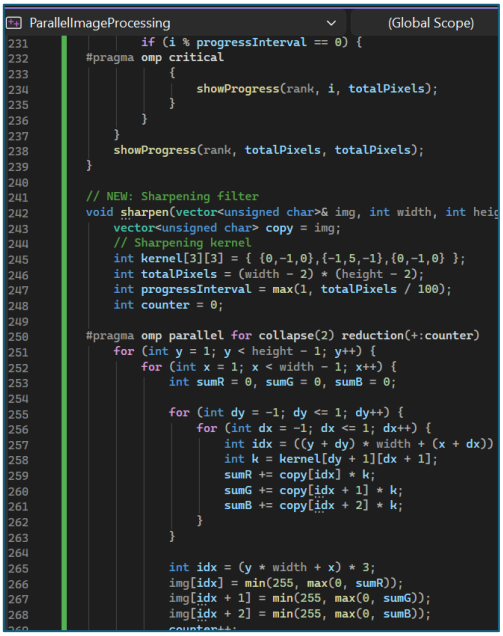



ParallelImageProcessing
(Global Scope)


ParallelImageProcessing
(Global Scope)


ParallelImageProcessing
(Global Scope)


ParallelImageProcessing
(Global Scope)


ParallelImageProcessing
(Global Scope)


ParallelImageProcessing
(Global Scope)



```
#pragma omp parallel for schedule(static)
    for (int i = 0; i < totalPixels; i++) {
        int idx = i * 3;

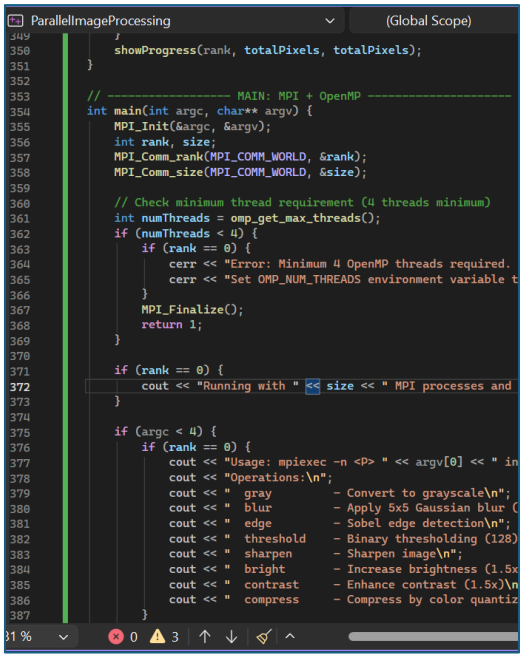
        for (int c = 0; c < 3; c++) {
            float pixel = img[idx + c];
            float adjusted = ((pixel - midpoint) * factor) + m
            img[idx + c] = min(255, max(0, static_cast<int>(ad
        }

        if (i % progressInterval == 0) {
#pragma omp critical
            {
                showProgress(rank, i, totalPixels);
            }
        }
    }
    showProgress(rank, totalPixels, totalPixels);
}

// NEW: Image Compression (quality reduction by downsampling c
void compress(vector<unsigned char>& img, int width, int heigh
    int totalPixels = width * height;
    int progressInterval = max(1, totalPixels / 100);
    int divisor = 256 / levels;

#pragma omp parallel for schedule(static)
    for (int i = 0; i < totalPixels; i++) {
        int idx = i * 3;
        img[idx] = (img[idx] / divisor) * divisor;
        img[idx + 1] = (img[idx + 1] / divisor) * divisor;
        img[idx + 2] = (img[idx + 2] / divisor) * divisor;

        if (i % progressInterval == 0) {
#pragma omp critical
            {
                showProgress(rank, i, totalPixels);
            }
```



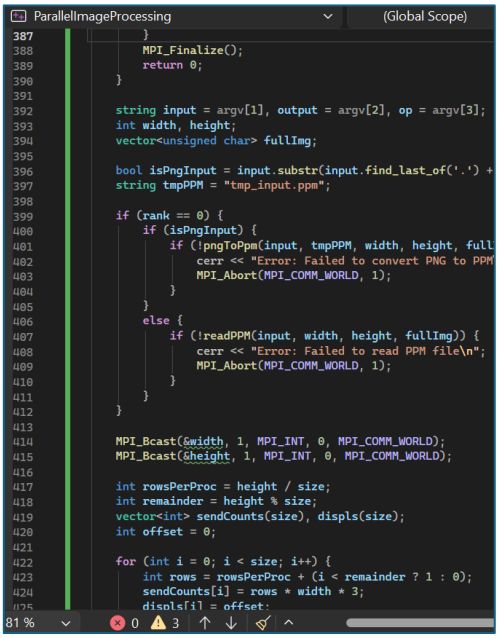
```
    showProgress(rank, totalPixels, totalPixels);
}

// ------------------ MAIN: MPI + OpenMP ------------------
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Check minimum thread requirement (4 threads minimum)
    int numThreads = omp_get_max_threads();
    if (numThreads < 4) {
        if (rank == 0) {
            cerr << "Error: Minimum 4 OpenMP threads required.
            cerr << "Set OMP_NUM_THREADS environment variable t
        }
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        cout << "Running with " << size << " MPI processes and
    }

    if (argc < 4) {
        if (rank == 0) {
            cout << "Usage: mpiexec -n <P> " << argv[0] << " in
            cout << "Operations:\n";
            cout << "  gray        - Convert to grayscale\n";
            cout << "  blur        - Apply 5x5 Gaussian blur (
            cout << "  edge        - Sobel edge detection\n";
            cout << "  threshold   - Binary thresholding (128)
            cout << "  sharpen     - Sharpen image\n";
            cout << "  bright      - Increase brightness (1.5x
            cout << "  contrast    - Enhance contrast (1.5x)\n
            cout << "  compress    - Compress by color quantiz
        }
```



```
        }
        MPI_Finalize();
        return 0;
    }

    string input = argv[1], output = argv[2], op = argv[3];
    int width, height;
    vector<unsigned char> fullImg;

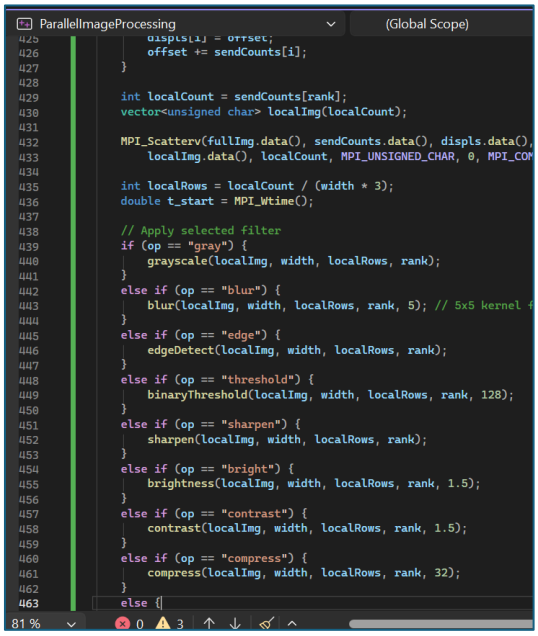
    bool isPngInput = input.substr(input.find_last_of('.') +
    string tmpPPM = "tmp_input.ppm";

    if (rank == 0) {
        if (isPngInput) {
            if (!pngToPpm(input, tmpPPM, width, height, full
                cerr << "Error: Failed to convert PNG to PPM
                MPI_Abort(MPI_COMM_WORLD, 1);
            }
        }
        else {
            if (!readPPM(input, width, height, fullImg)) {
                cerr << "Error: Failed to read PPM file\n";
                MPI_Abort(MPI_COMM_WORLD, 1);
            }
        }
    }

    MPI_Bcast(&width, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&height, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int rowsPerProc = height / size;
    int remainder = height % size;
    vector<int> sendCounts(size), displs(size);
    int offset = 0;

    for (int i = 0; i < size; i++) {
        int rows = rowsPerProc + (i < remainder ? 1 : 0);
        sendCounts[i] = rows * width * 3;
        displs[i] = offset;
```



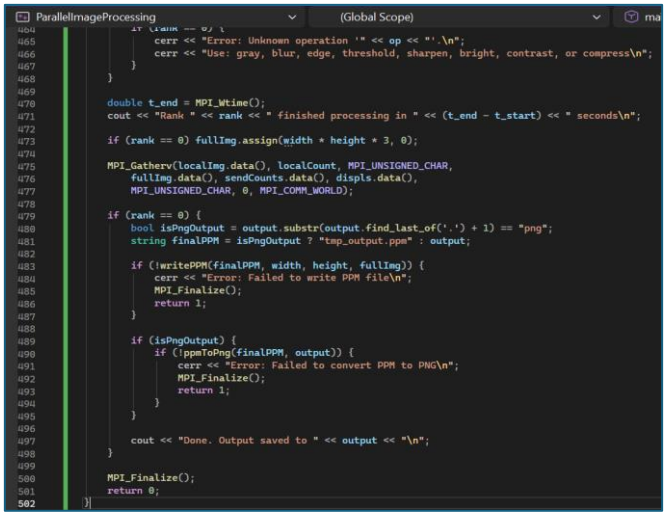
```
        offset += sendCounts[i];
    }

    int localCount = sendCounts[rank];
    vector<unsigned char> localImg(localCount);

    MPI_Scatterv(fullImg.data(), sendCounts.data(), displs.data(),
        localImg.data(), localCount, MPI_UNSIGNED_CHAR, 0, MPI_COM

    int localRows = localCount / (width * 3);
    double t_start = MPI_Wtime();

    // Apply selected filter
    if (op == "gray") {
        grayscale(localImg, width, localRows, rank);
    }
    else if (op == "blur") {
        blur(localImg, width, localRows, rank, 5); // 5x5 kernel f
    }
    else if (op == "edge") {
        edgeDetect(localImg, width, localRows, rank);
    }
    else if (op == "threshold") {
        binaryThreshold(localImg, width, localRows, rank, 128);
    }
    else if (op == "sharpen") {
        sharpen(localImg, width, localRows, rank);
    }
    else if (op == "bright") {
        brightness(localImg, width, localRows, rank, 1.5);
    }
    else if (op == "contrast") {
        contrast(localImg, width, localRows, rank, 1.5);
    }
    else if (op == "compress") {
        compress(localImg, width, localRows, rank, 32);
    }
    else {
```



```
            cerr << "Error: Unknown operation '" << op << "'.\n";
            cerr << "Use: gray, blur, edge, threshold, sharpen, bright, contrast, or compress\n";
        }
    }

    double t_end = MPI_Wtime();
    cout << "Rank " << rank << " finished processing in " << (t_end - t_start) << " seconds\n";

    if (rank == 0) fullImg.assign(width * height * 3, 0);

    MPI_Gatherv(localImg.data(), localCount, MPI_UNSIGNED_CHAR,
        fullImg.data(), sendCounts.data(), displs.data(),
        MPI_UNSIGNED_CHAR, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        bool isPngOutput = output.substr(output.find_last_of('.') + 1) == "png";
        string finalPPM = isPngOutput ? "tmp_output.ppm" : output;

        if (!writePPM(finalPPM, width, height, fullImg)) {
            cerr << "Error: Failed to write PPM file\n";
            MPI_Finalize();
            return 1;
        }

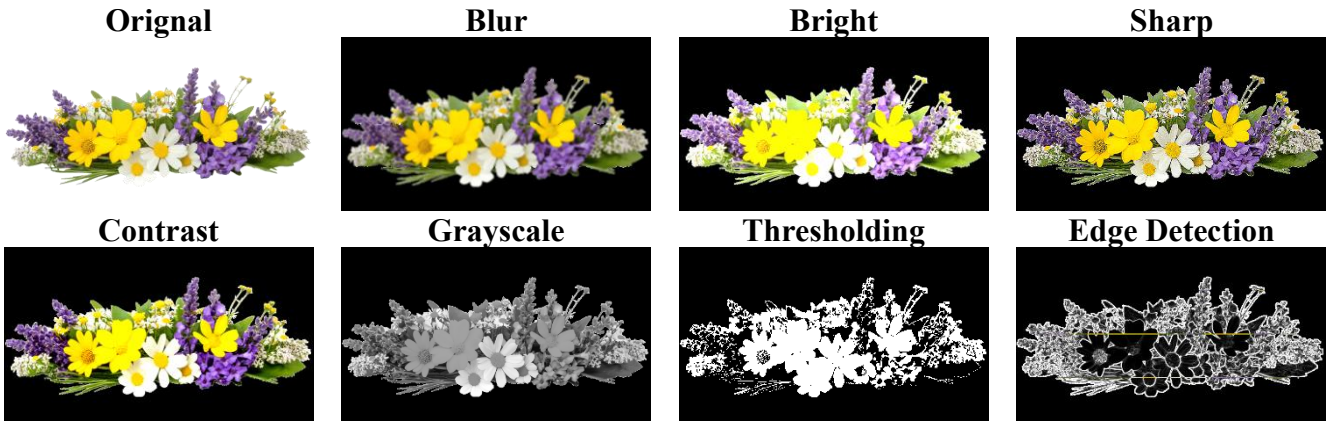
        if (isPngOutput) {
            if (!ppmToPng(finalPPM, output)) {
                cerr << "Error: Failed to convert PPM to PNG\n";
                MPI_Finalize();
                return 1;
            }
        }

        cout << "Done. Output saved to " << output << "\n";
    }

    MPI_Finalize();
    return 0;
}
```

Output:

Below are the sample outputs from program:



Conclusion:

This project successfully demonstrates that the hybrid MPI + OpenMP architecture significantly accelerates image processing by combining distributed and shared-memory parallelism. MPI enabled efficient distribution of image segments across multiple processes or nodes, while OpenMP provided fast, shared-memory parallelism within each process. Together, these techniques reduced execution time, improved resource utilization, and allowed large image files to be processed far more efficiently than in a purely sequential approach. The project confirms that hybrid parallelism is far more effective than using MPI or OpenMP alone, especially for pixel-intensive operations on large images.

Future Work

The system can be extended to include

1. **GPU Acceleration:** Use CUDA/OpenCL for faster, real-time filter processing.
2. **More Image Formats:** Add support for JPEG, TIFF, BMP, and RAW.
3. **Pipeline Parallelism:** Process multiple filters in a continuous multi-stage pipeline.
4. **Distributed Storage:** Integrate HDFS or cloud storage for large-scale datasets.
5. **User Interface:** Provide a GUI for easy image upload, filter selection, and visualization.